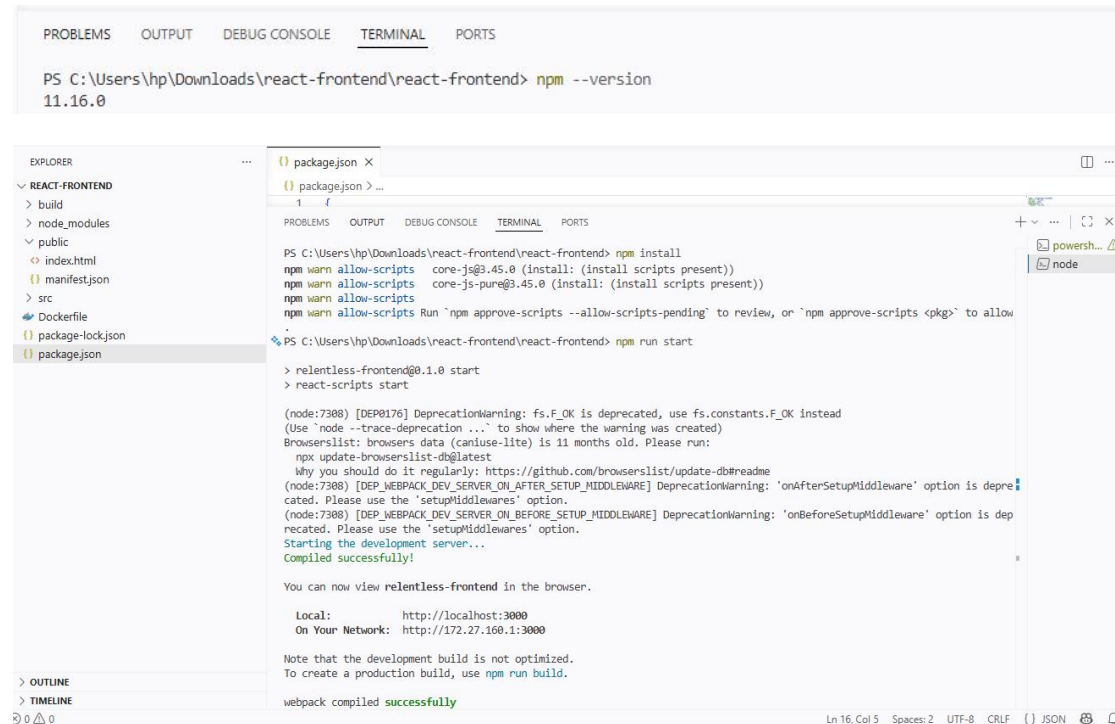


winget install OpenJS.NodeJS.LTS ---to install runtime javascript(NodeJS)
npm install ---to install application dependencies(package.json)
npm start --run application

Application running without docker:



```
PS C:\Users\hp\Downloads\react-frontend\react-frontend> npm --version
11.16.0

PS C:\Users\hp\Downloads\react-frontend\react-frontend> npm install
npm warn allow-scripts core-js@3.45.0 (install: (install scripts present))
npm warn allow-scripts core-js-pure@3.45.0 (install: (install scripts present))
npm warn allow-scripts Run 'npm approve-scripts --allow-scripts-pending' to review, or 'npm approve-scripts <pkg>' to allow

PS C:\Users\hp\Downloads\react-frontend\react-frontend> npm run start

> relentless-frontend@0.1.0 start
> react-scripts start

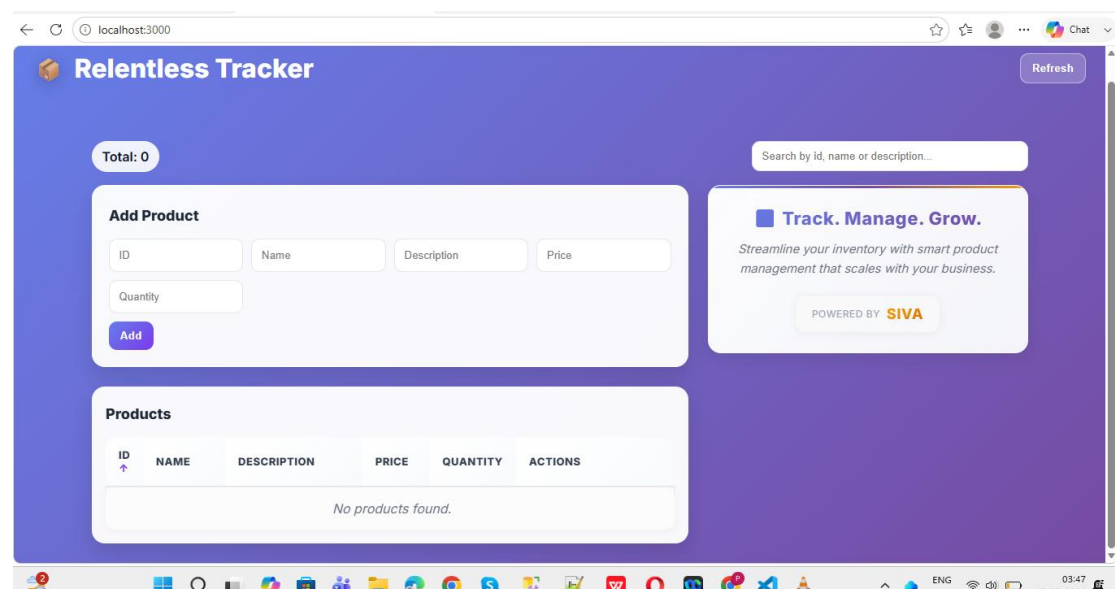
(node:7388) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.F_OK instead
(Use 'node --trace-deprecation ...' to show where the warning was created)
Browserslist: browsers data (caniuse-lite) is 11 months old. Please run:
  npx update-browserslist-db@latest
  Why you should do it regularly: https://github.com/browserslist/update-db#readme
(node:7388) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: 'onAfterSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
(node:7388) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning: 'onBeforeSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' option.
Starting the development server...
Compiled successfully!

You can now view relentless-frontend in the browser.

Local:      http://localhost:3000
On Your Network:  http://172.27.168.1:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

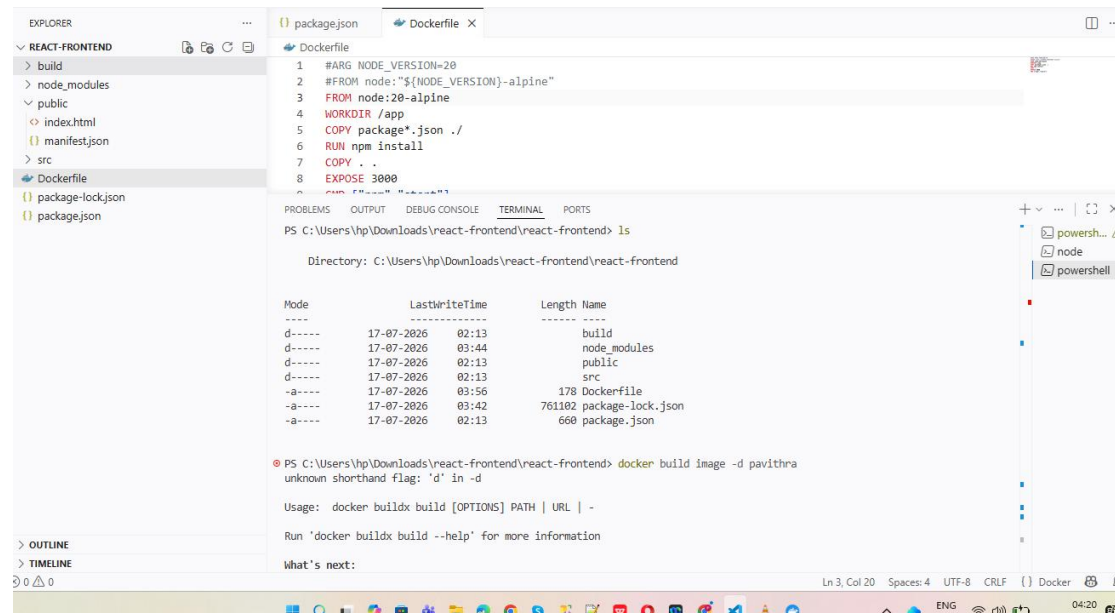
webpack compiled successfully
```



docker build -t imagename .
docker images
Docker run -d -p 3000:3000 pavithra

docker ps

Application running using docker :



The screenshot shows a VS Code editor with a file explorer on the left displaying the project structure: `REACT-FRONTEND` (containing `build`, `node_modules`, `public`, `index.html`, `manifest.json`, `src`, `Dockerfile`, `package-lock.json`, and `package.json`), and a `Dockerfile` file open in the editor. The `Dockerfile` contains the following instructions:

```
1 #ARG NODE_VERSION=20
2 #FROM node:${NODE_VERSION}-alpine
3 FROM node:20-alpine
4 WORKDIR /app
5 COPY package*.json ./
6 RUN npm install
7 COPY . .
8 EXPOSE 3000
```

The terminal window shows the output of the `ls` command in the directory `C:\Users\hp\Downloads\react-frontent\react-frontent`:

```
PS C:\Users\hp\Downloads\react-frontent\react-frontent> ls
Directory: C:\Users\hp\Downloads\react-frontent\react-frontent

Mode                LastWriteTime         Length Name
----                -
d-----            17-07-2026    02:13         build
d-----            17-07-2026    03:44       node_modules
d-----            17-07-2026    02:13         public
d-----            17-07-2026    02:13         src
-a-----            17-07-2026    03:56         178 Dockerfile
-a-----            17-07-2026    03:42       761102 package-lock.json
-a-----            17-07-2026    02:13         660 package.json
```

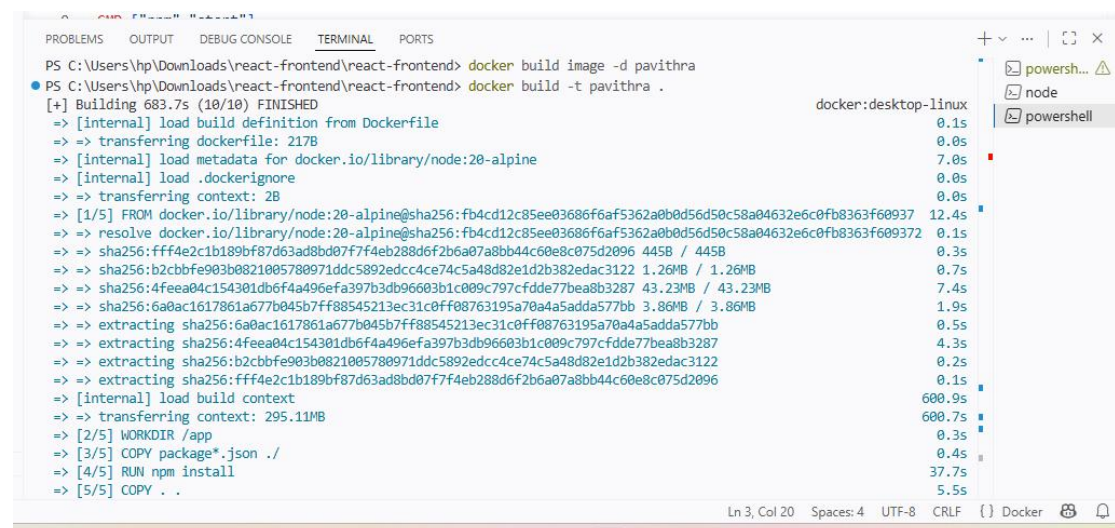
Below the file listing, the terminal shows an error message:

```
PS C:\Users\hp\Downloads\react-frontent\react-frontent> docker build image -d pavithra
unknown shorthand flag: 'd' in -d

Usage: docker buildx build [OPTIONS] PATH | URL | -

Run 'docker buildx build --help' for more information

What's next:
```



The screenshot shows the terminal window with the output of the `docker build -t pavithra .` command. The build process is successful, and the image is ready to be used.

```
PS C:\Users\hp\Downloads\react-frontent\react-frontent> docker build image -d pavithra
PS C:\Users\hp\Downloads\react-frontent\react-frontent> docker build -t pavithra .
[+] Building 683.7s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 217B
=> [internal] load metadata for docker.io/library/node:20-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372
=> => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372
=> => sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096 445B / 445B
=> => sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122 1.26MB / 1.26MB
=> => sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287 43.23MB / 43.23MB
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb
=> => extracting sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287
=> => extracting sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122
=> => extracting sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096
=> [internal] load build context
=> => transferring context: 295.11MB
=> [2/5] WORKDIR /app
=> [3/5] COPY package*.json ./
=> [4/5] RUN npm install
=> [5/5] COPY . .
```



The screenshot shows the terminal window with the output of the `docker images` command, listing the built images:

```
PS C:\Users\hp\Downloads\react-frontent\react-frontent> docker images
IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
nginx:latest   b5a9a3cfc86b      241MB       66MB          U
pavithra:latest 21d6821dff99      1.29GB      215MB
```

Below the image listing, the terminal shows the output of the `docker run -d -p 3000:3000 pavithra` command, which successfully starts the container:

```
PS C:\Users\hp\Downloads\react-frontent\react-frontent> docker run -d -p 3000:3000 pavithra
b4deba3a59ebc016c36cd7bb027a48dafbc194690003a096f382bad7097f2487
PS C:\Users\hp\Downloads\react-frontent\react-frontent> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
b4deba3a59eb  pavithra      "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
cp            focused_clarke
```

Finally, the terminal shows the output of the `docker ps` command, which lists the running container:

```
PS C:\Users\hp\Downloads\react-frontent\react-frontent> docker ps
```

docker.desktopPERSONAL

SearchCtrl+K

Sign in

Containers

Images

Logs

Volumes

Kubernetes

Builds

Models

MCP ToolkitBETA

Docker Hub

Extensions

Containers

Give feedback

Container CPU usage0.09% / 800% (8 CPUs available)

Container memory usage195.1MB / 7.45GB

Show charts

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last	Actions
	epic_spence	2b73142a48c8	nginx	8080:80	0%	2 da	
	focused_clarke	b4deba3a59eb	pavithra	3000:3000	0.13%	7 mi	

Walkthroughs

Multi-container applications

8 mins

Containerize your application

3 mins

View more in the Learning center

Engine runningRAM 4.95 GB CPU 0.13% Disk: 4.20 GB used (limit 1006.85 GB)v4.82.0

Relentless Tracker

Refresh

Total: 0

Search by id, name or description...

Add Product

ID

Name

Description

Price

Quantity

Add

Products

ID	NAME	DESCRIPTION	PRICE	QUANTITY	ACTIONS
No products found.					

Track. Manage. Grow.

Streamline your inventory with smart product management that scales with your business.

POWERED BY SIVA